

Traffic Simulator - Part 3

Carried out within the scope of the "*Telecommunications Networks*" course.

André Silva Leonor Guedes Rafael Parody
up202107419 up202107691 up202502656

The source code developed for this lab work is available at:

https://github.com/mLeOnOr/Traffic_Simulator/tree/main/lab1/Part3.

1 General characterization of the simulation methodology

This work aimed to simulate a Call Center system, using a discrete event simulation framework, in which the temporal evolution is represented exclusively through the chronological processing of discrete events. This way, instead of time progressing continuously, the simulation clock is advanced to the timestamp of the next scheduled event. To obtain statistically stable performance results, the simulation was run until a total of 100 000 call arrivals had been generated.

Calls arrive to the system according to a Poisson process with a rate $\lambda = 80$ calls/hour, and each incoming call is classified either as general-purpose or area-specific, with a probability of occurrence of 30% and 70%, respectively. Under this structure, the simulated call center consists of two subsystems: a **general-purpose answering system**, responsible for receiving all incoming calls, and an **area-specific answering system**, which handles the calls that require specialised assistance.

The behaviour of the system is strongly determined by the service-time distributions associated with each type of call. General-purpose calls can be entirely answered by a general-case operator and have a minimum duration of 1 minute, to which an exponentially distributed component with mean 2 minutes is added, making the maximum total duration 5 minutes. Area-specific calls first need to be answered by a general-case operator, who later transfers it to the area-specific subsystem. In this first stage these calls follow a Gaussian distribution, with a mean of 1 minute, a standard deviation of 20 seconds, and strict lower and upper bounds of 30 and 120 seconds, respectively. Once transferred to the area-specific subsystem, the remaining duration is again composed of a minimum of 1 minute plus an exponentially distributed component with mean 2.5 minutes.

In this discrete event representation, two classes of events are defined: **arrival events** and **departure events**. Upon the occurrence of each arrival event, and if the total number of calls to be simulated has not yet been reached, the next arrival is immediately scheduled, regardless of whether the incoming call will be served immediately or placed in the waiting queue. The timestamp of this next scheduled arrival is obtained by adding to the current simulation time a random inter-arrival interval generated according to an exponential distribution with parameter λ , which characterises the Poisson arrival process. If a general-purpose operator is available at the moment of arrival, the call is immediately answered. In this case, a departure event is scheduled at the current time plus the service duration generated according to the appropriate distribution. If all general-purpose operators are busy, the call is placed in the corresponding finite FIFO waiting queue, while the queue has not reached its maximum capacity.

When a departure event occurs in the general-purpose subsystem and the queue is not empty, the earliest waiting call immediately enters service, and a new departure event is scheduled for its completion time. Independently of the queue state, if the departure call is area-specific it is forwarded to the area-specific subsystem. This subsystem follows a similar event-driven logic, if an area-specific operator is available, the call begins service immediately, and a departure event is scheduled according to the service-time distribution defined for this stage. If all operators in this subsystem are busy, the call joins an infinite FIFO waiting queue. Departure events are processed analogously: if the queue is not empty, the next event is removed and a new departure event is scheduled. The following image illustrates the generation of events process.

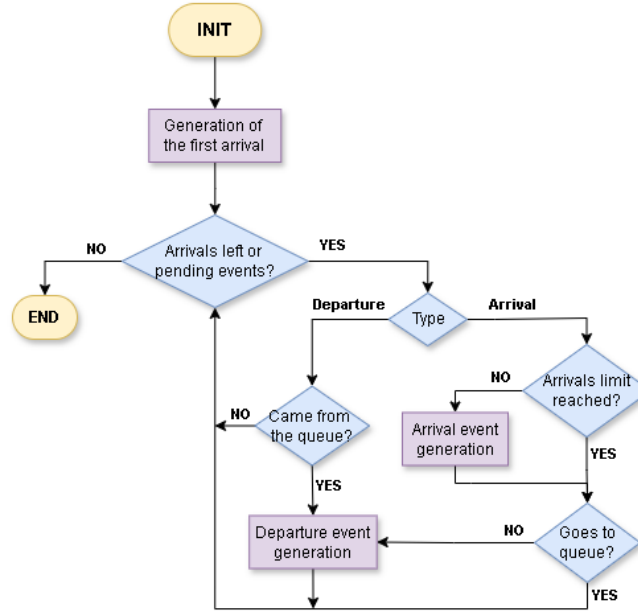


Figure 1: Event generation flow diagram.

2 Description of the simulation program

Building on the events structure and subsystems interactions described in the previous section, the simulation program organises the system's evolution through two time-ordered event lists, one for each subsystem. At each iteration, the simulation clock advances to whichever event is scheduled to occur next in the overall system. Since the lists are independent, the program inspects the first element of each list, which corresponds to the earliest pending event in each subsystem. Their timestamps are compared, and the event with the smaller timestamp is selected for processing, ensuring that the two subsystems operate together as a single consistent system. If one of the lists is empty, a large value is assigned to it so that the next event is selected from the non-empty list.

At the beginning of the simulation, the program generates the first arrival according to the Poisson arrival model. The main simulation loop then continues while the predefined number of arrivals has not yet been generated or while pending events remain in either list.

In the general-purpose subsystem, an arriving call enters service immediately whenever at least one general-purpose operator is free. If all operators are busy, the call is placed in the finite waiting queue, as long as the queue has not reached its maximum capacity L . A call is therefore considered delayed whenever it must wait in this queue before entering service. If both the operators and the queue are full at the moment of arrival, the call cannot be answered and is blocked. With this, the delay probability is given by the ratio between the number of delayed calls and the total number of arrivals. Similarly, the blocking probability corresponds to the ratio between the number of blocked calls and the total number of arrivals.

When the next event corresponds to the departure of an area-specific call, the call is forwarded to the area-specific subsystem by being added to its event list. In this subsystem, the call can either be answered immediately, if there is any area-specific operator free, or placed in the infinite waiting queue. Along with the call, the simulation forwards the timestamp at which it originally entered the general-purpose subsystem. This information allows the program to compute, at the moment the departure occurs, the total time elapsed between the call's arrival to the system and the completion of the specialised service.

To clarify the sequence of decisions and event transitions described above, the corresponding flow diagram is presented in Figure 2.

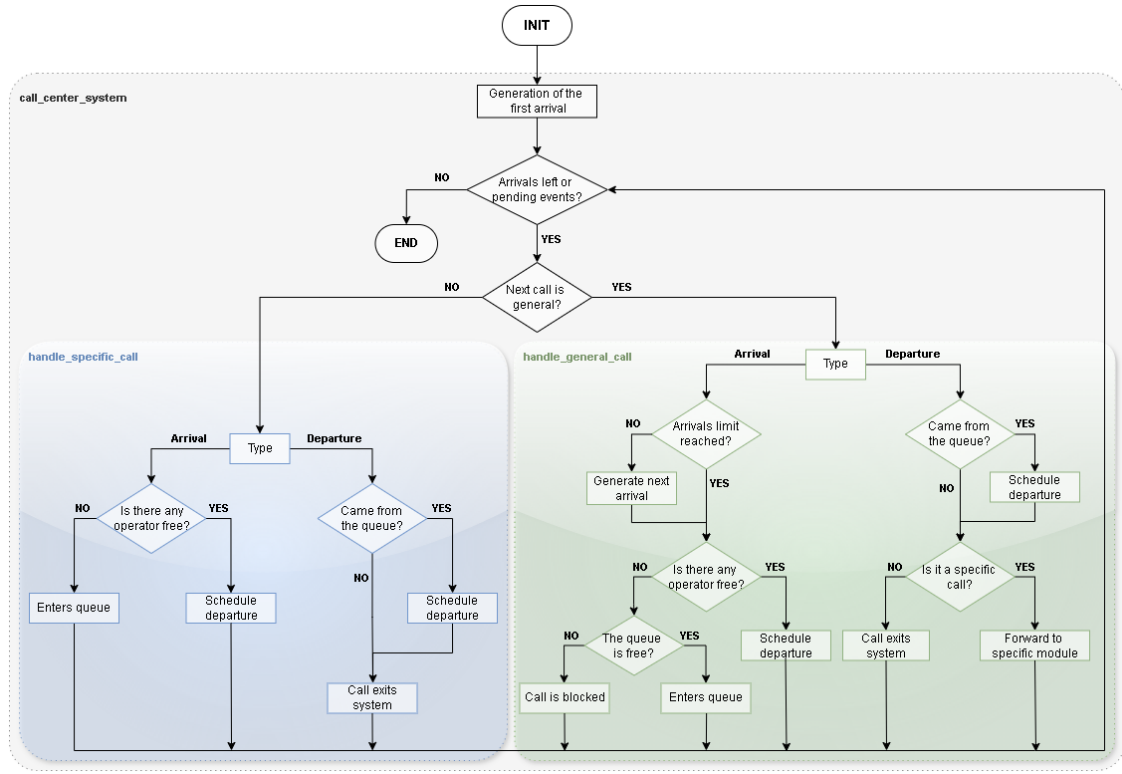


Figure 2: Simulation flow diagram.

3 Waiting time prediction algorithm

The simulation incorporates a simple predictive mechanism for estimating the waiting time of calls in the general-purpose queue. The prediction is based on a running average of the actual waiting times experienced by the calls that have already entered service. Each time a call leaves the queue, its waiting time is computed as the difference between the current time and its arrival timestamp. This value is then incorporated into a cumulative average, updated incrementally throughout the simulation. This running average represents the current estimate of the expected waiting time per queued call.

Whenever a call goes to the waiting queue, the predicted waiting time is given by the current running average multiplied by the number of calls ahead in the queue. To evaluate the accuracy of this prediction, the simulation stores both the predicted and the actual waiting time of each delayed call. The absolute prediction error is computed as the absolute difference between the predicted and actual waiting times, while the relative prediction error is obtained by dividing this difference by the actual waiting time.

4 Simulation results

The simulation evaluated multiple configurations of the call centre, defined by the number of general-purpose (N_g) and area-specific (N_s) operators, and the length (L) of the general-purpose waiting queue. For each configuration, main performance metrics were obtained: delay and blocking probability, average delay, absolute and relative prediction errors and the average time between the arrival of a call to the general-purpose answering system and the call answering by the area-specific answering system. These results were then used to determine the configuration that best meets the minimal performance goals of the system.

To identify the best configuration, each simulated scenario was evaluated using a score function that rewards good performance while penalising excessive resource usage, thus balancing service quality with operational cost. To calculate the score, three performance metrics were considered: the blocking probability, the average waiting time in the queue, and the average total time that calls spend in the system. Their values were normalised with respect to the maximum acceptable limits, and each account for 20% of the score. The remaining 40% corresponds to the total number of operators required by the configuration, normalised by the maximum number of operators simulated. This higher weight ensures that the score favours configurations that provide acceptable performance without relying on excessive staffing. The simulation results were also inspected to identify the scenario whose performance indicators were closest to the established thresholds. The results obtained are as follows:

Closest-to-limit: 4 general operators,
queue length 2, 5 area-specific operators

Delay probability:	15.14%	($\leq 30\%$)
Blocking probability:	1.95%	($\leq 2\%$)
Average delay:	26.01 s	($\leq 30s$)
Prediction absolute error:	19.92 s	
Prediction relative error:	0.90	
Average total time:	85.60 s	($\leq 90s$)

Best combination: 6 general operators,
queue length 2, 6 area-specific operators

Delay probability:	1.71%	($\leq 30\%$)
Blocking probability:	0.11%	($\leq 2\%$)
Average delay:	16.46 s	($\leq 30s$)
Prediction absolute error:	13.59 s	
Prediction relative error:	0.94	
Average total time:	70.03 s	($\leq 90s$)

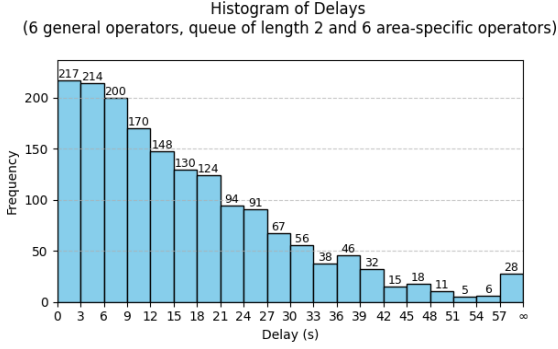


Figure 3: Histogram of delays.

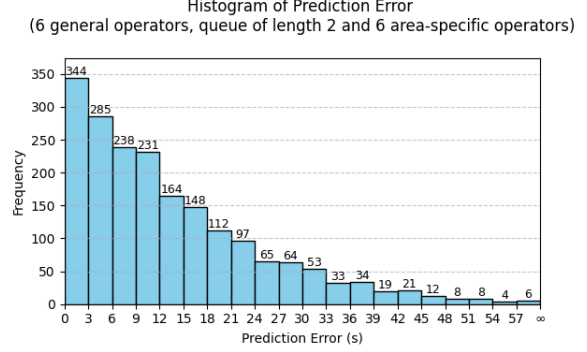


Figure 4: Histogram of prediction errors.

With an arrival rate of $\lambda = 80$ calls/h (≈ 1.33 calls/min) and an average general-purpose service time $E[S_g] = 0.30 \times 3 \text{ min} + 0.70 \times 1 \text{ min} = 1.6 \text{ min}$, the offered traffic to the general-purpose subsystem is $A_g = \lambda \times E[S_g] \approx 1.33 \times 1.6 = 2.13$ erlangs. Since stability requires $\rho = A_g/N_g < 1$, at least $N_g \geq 3$ operators are needed. For area-specific calls, only 70% of arrivals are forwarded, giving an effective arrival rate of $\lambda_s = 0.7 \times \lambda \approx 0.93$ calls/min. Using the corresponding average specialised service time, the offered traffic becomes $A_s = \lambda_s \times E[S_s] \approx 0.93 \times 3.5 = 3.27$ erlangs, which implies stability for $N_s \geq 4$. Consistently with this theoretical analysis, the simulations showed that configurations with $N_g < 3$ or $N_s < 4$ produced very high average total delay, confirming system saturation in those cases.

5 Sensitivity analysis

To evaluate how the average total delay reacts to variations in the offered traffic, the simulation was repeated 30 times for a range of arrival rates around the nominal value of 80 calls/hour. In each run, a running average estimator was used to obtain a single estimate of the total delay, measured from the arrival of a call at the general-purpose subsystem until its completion in the area-specific subsystem.

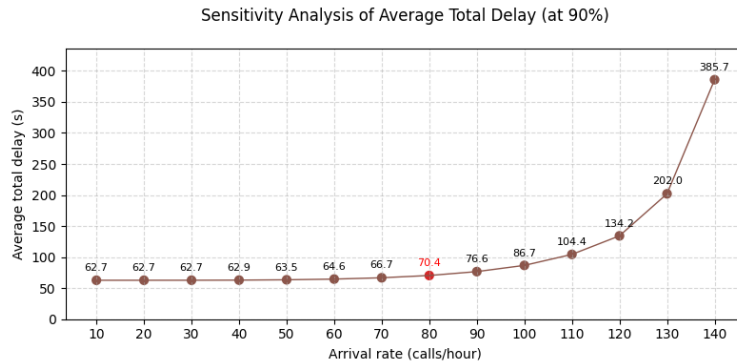


Figure 5: Sensitivity analysis of the average total delay.

The sensitivity curve (Figure 5) shows that the average total delay remains almost flat for arrival rates up to about 70 calls/hour, while near the nominal rate of 80 calls/hour, the delay begins to rise. Beyond this point the growth becomes rapid, reaching more than 200 seconds at 130 calls/hour and exploding to nearly 400 seconds at 140 calls/hour. This behaviour confirms that the selected configuration operates comfortably below saturation for moderate loads, but becomes increasingly sensitive as the arrival rate approaches the system's effective capacity limit.